NoteFlow – Initial Design (with UML)

Ahmar Usman ID: 1751130
Mohammad AlSaiad ID: 1750755
Yazan Al Rifaee ID: 1751157

1. Design Overview

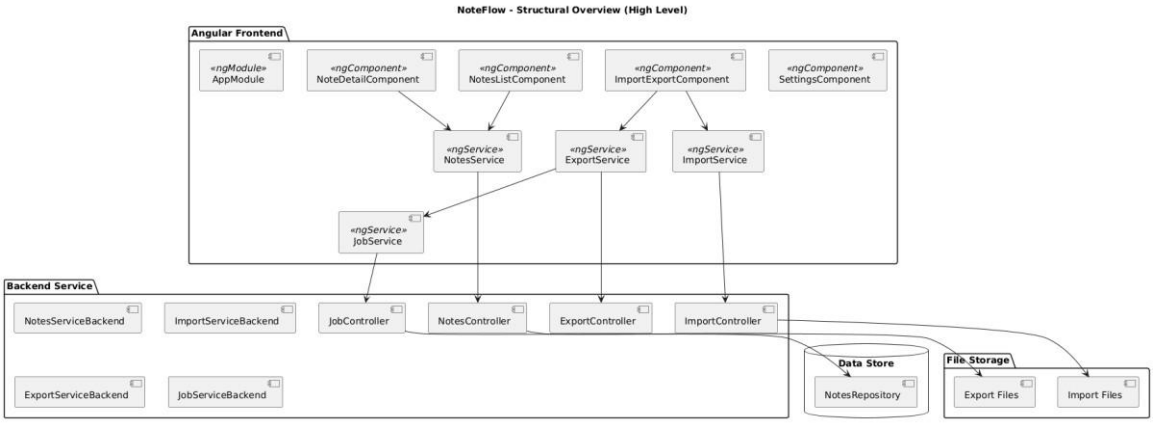
This document describes the initial design of the NoteFlow – Notes & Export Tool system. The design provides a high-level overview of the system structure and behavior.

2. System Architecture

The system follows a layered architecture consisting of an Angular-based frontend and a backend service. The frontend handles user interaction, routing, and presentation logic, while the backend is responsible for data persistence, file import and export, and longrunning processing tasks.

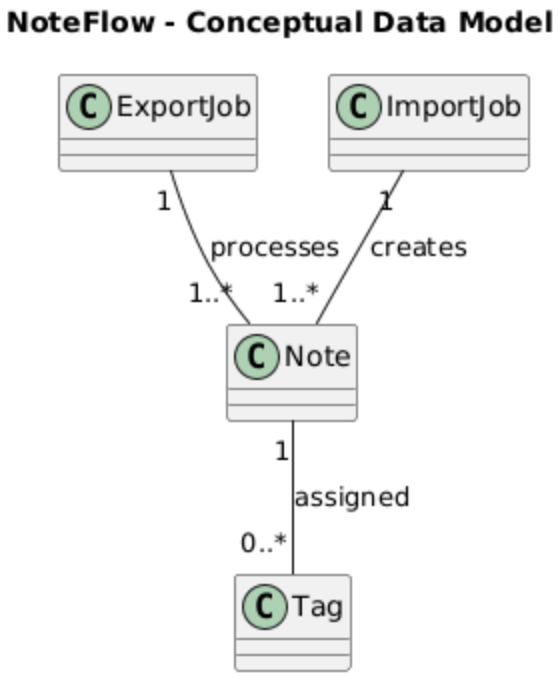
3. UML Structural Diagram

The following UML diagram shows the main frontend and backend elements at a high level.



3.1 Conceptual Data Model

The system manages different types of data entities that are required to support note management and long-running import and export processes. The following model illustrates the main data objects and their relationships at a high level. The diagram intentionally omits attributes and implementation details.



4. Frontend Design

4.1 Routing Overview

The frontend uses the Angular Router to navigate between application views. The main routes of the application are listed below.

Route	Component	Purpose
/notes	NotesListComponent	List and search notes
/notes/:id	NoteDetailComponent	View and edit a note
/import-export	ImportExportComponent	Import and export notes

4.2 Component Overview

The frontend is composed of the following main components:

Component	Responsibility
NotesListComponent	Displays and filters notes
NoteDetailComponent	Creates and edits notes
ImportExportComponent	Triggers import and export processes

/settings	SettingsComponent	User settings
SettingsComponent	Manages user preferences	

5. Backend Design

5.1 API Overview

The backend exposes RESTful endpoints to support frontend operations.

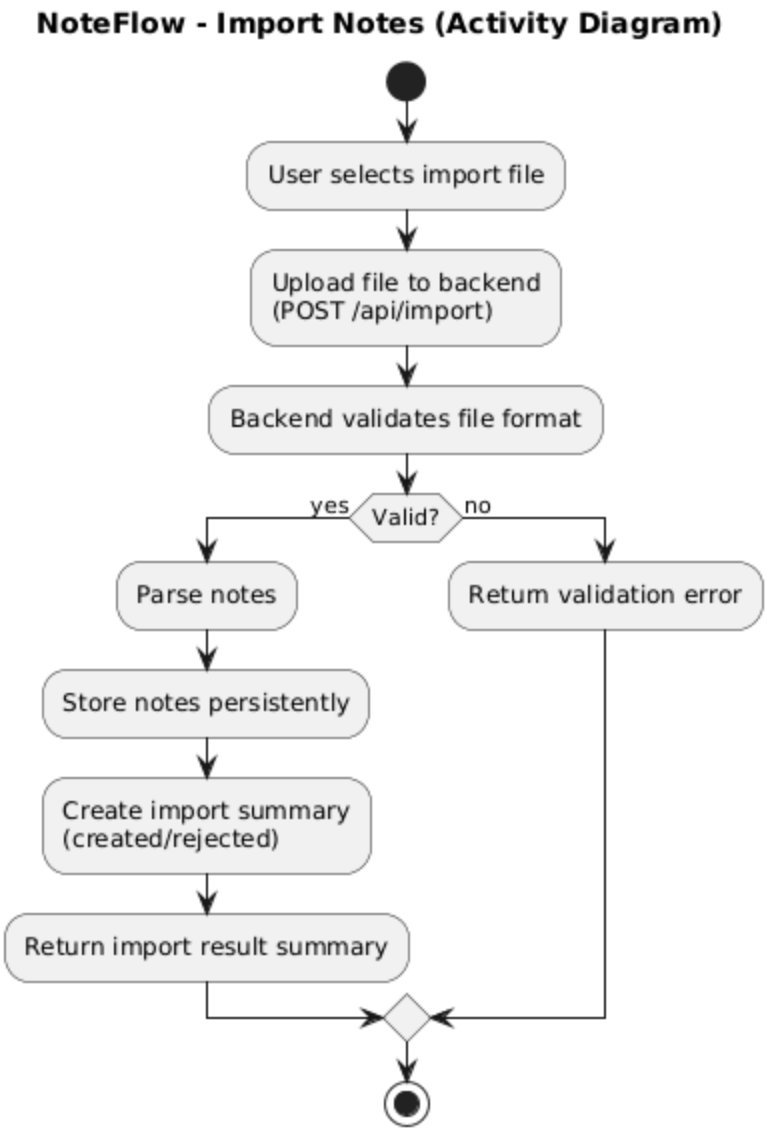
Endpoint	Method	Purpose
/api/notes	GET	Retrieve notes
/api/notes	POST	Create note
/api/notes/{id}	PUT	Update note
/api/notes/{id}	DELETE	Delete note
/api/import	POST	Import notes
/api/export	POST	Start export job
/api/jobs/{id}	GET	Retrieve job status
/api/export/{id}	GET	Download export file

5.2 Long-Running Export Process

When a user requests an export, the backend creates an asynchronous export job. The job processes the notes and periodically updates its progress. The frontend polls the job status and displays the progress until the job is completed.

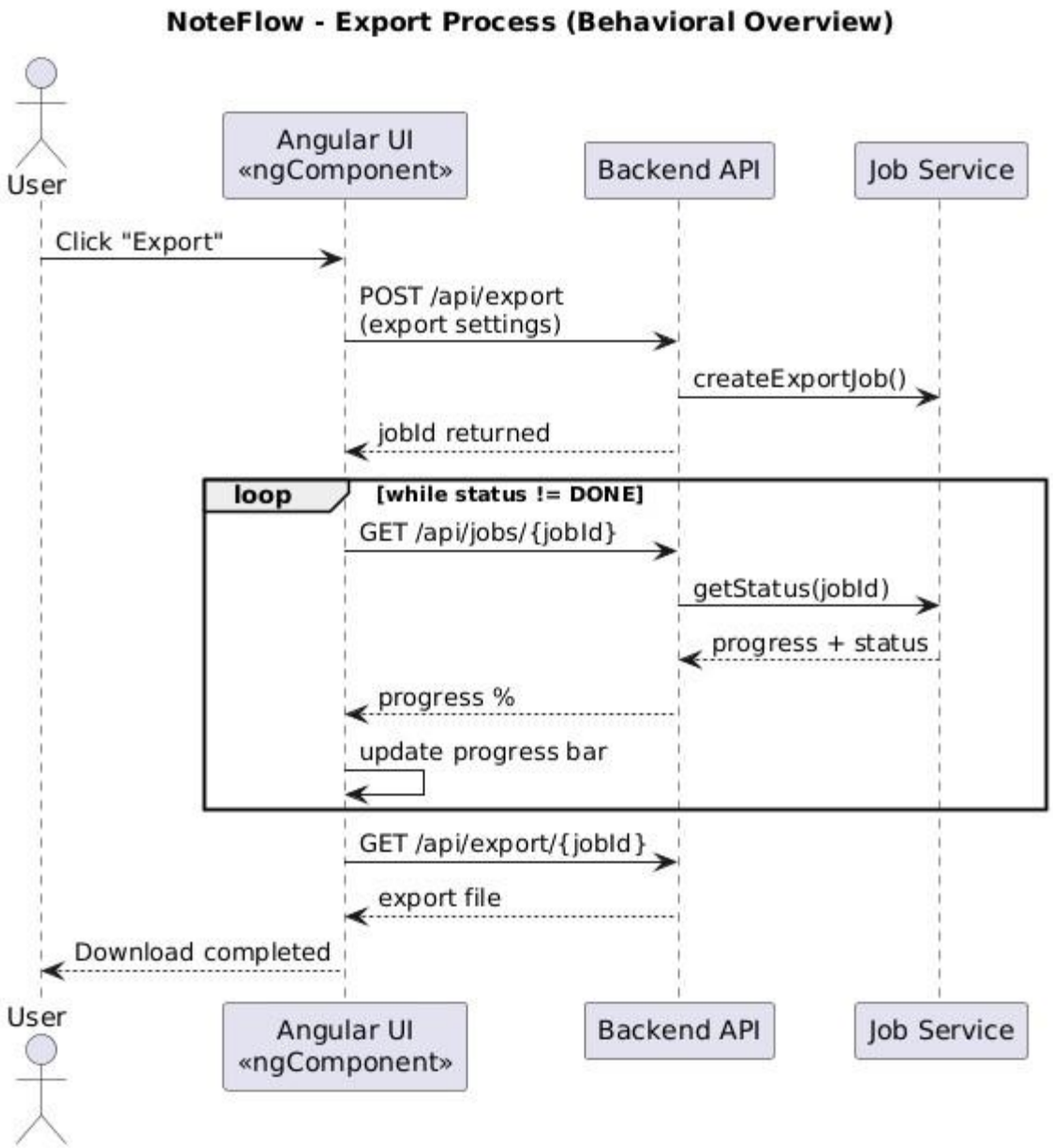
5.3 Import Process Overview

In addition to export processing, the import functionality follows a similar asynchronous interaction pattern. When a user uploads a notes file, the backend validates the input and processes the import operation. The frontend receives status feedback and presents a summary of the import result once the process is completed.



6. UML Behavioral Diagram

The following UML sequence diagram illustrates the export process at a high level.



7. Traceability to Requirements

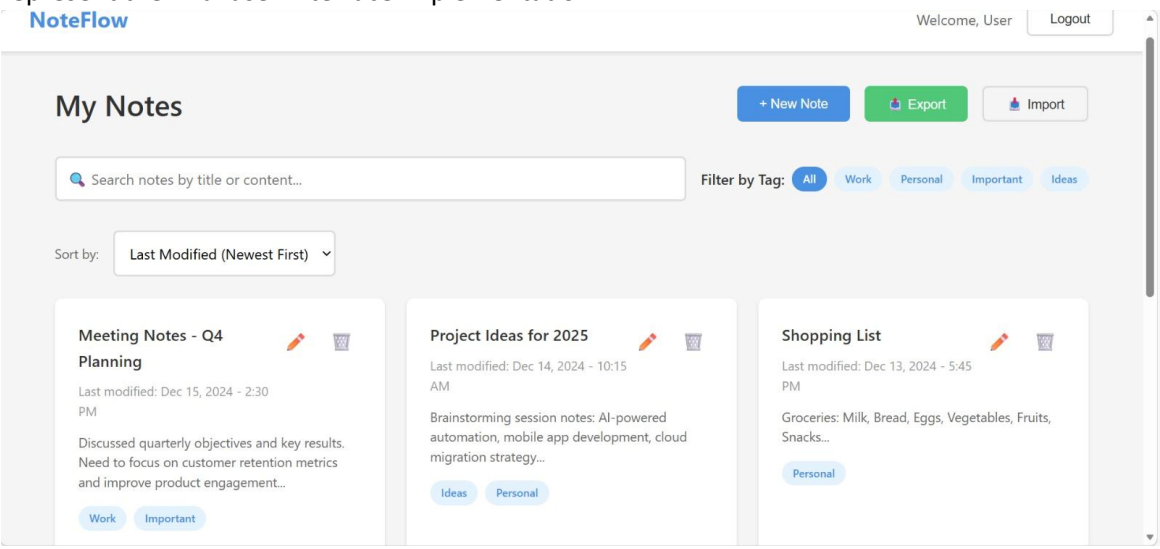
Key design elements are mapped to the requirements as follows:

Requirement	Design Element
R03–R11	NotesListComponent, NoteDetailComponent
R14–R16	Export job and progress polling
R18–R20	Import endpoint and validation

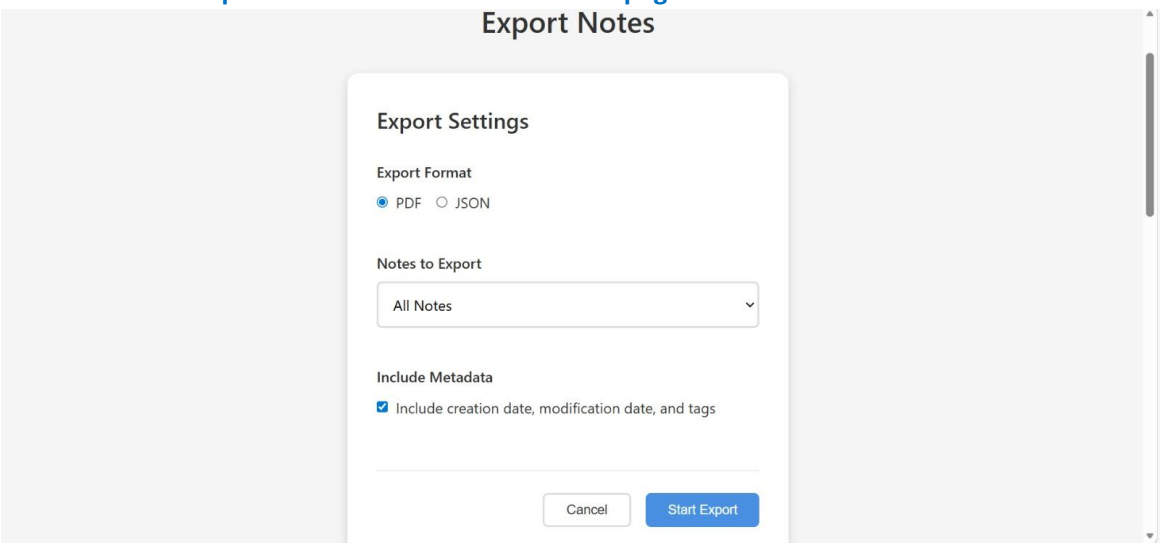
A01–A05	Angular frontend and test setup

Appendix A – UI Design Mocks

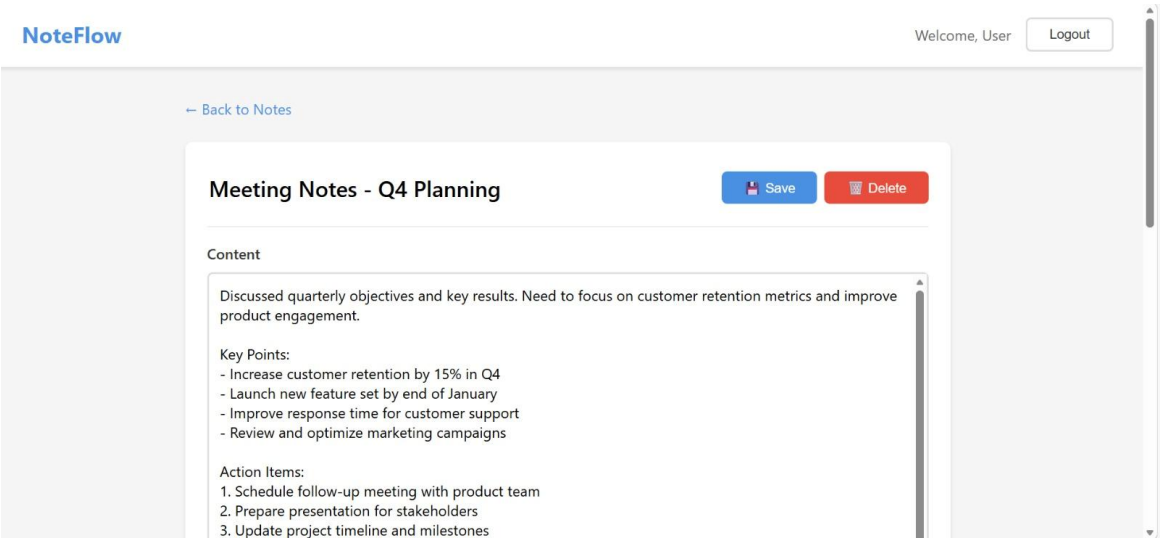
The following UI design mocks are based on early HTML/CSS sketches of the application. They are included as screenshots to illustrate layout and interaction concepts only and do not represent the final user interface implementation.



This screenshot represents a mock of the notes list page.



This screenshot represents a mock of the export functionality of a note.



This screenshot represents a mock of the creation of a note.